JAVA CORE & SPRING CORE

Tài liệu đề cương học tập và ôn phỏng vấn

Đề cương chi tiết theo mô hình 3 tầng: Core Internals → Production Engineering → Interview & System Reasoning

Đối tượng: người học Java Backend hướng tới vị trí Intern/Junior và muốn xây dựng nền tảng đủ sâu để tiếp tục phát triển.

Phạm vi: danh sách chủ đề và câu hỏi tự luyện; tài liệu chưa bao gồm đáp án.

Gợi ý phiên bản: dùng Java 21 làm baseline và đối chiếu phiên bản Spring thực tế của project khi đọc tài liệu.

Mục tiêu không phải học thuộc câu trả lời. Mục tiêu là giải thích được cơ chế, kiểm chứng bằng code và bảo vệ được lựa chọn kỹ thuật.

Mục lục

- Phần I. Phương pháp luyện trả lời theo khung năm bước
- Phần II. Cách sử dụng tài liệu và kế hoạch học
- Phần III. Java Core — đề cương chi tiết theo 3 tầng
- Phần IV. Spring Core — đề cương chi tiết theo 3 tầng
- Phần V. Bài tập cốt lõi và kế hoạch luyện phỏng vấn
- Phần VI. Ngân hàng 80 câu hỏi: 40 Java Core + 40 Spring Core
- Phần VII. Checklist tự đánh giá

Lưu ý: đây là tài liệu định hướng học tập. Các chủ đề có thể được điều chỉnh theo yêu cầu tuyển dụng, phiên bản Java/Spring và mức độ ưu tiên của từng vị trí.

PHẦN I — PHƯƠNG PHÁP LUYỆN TRẢ LỜI

Dùng khung năm bước cho mỗi câu hỏi. Hãy tự trả lời trước khi xem tài liệu hoặc tìm kiếm đáp án. Với câu hỏi đơn giản, câu trả lời có thể ngắn; với câu hỏi về JVM, transaction hoặc concurrency, cần giải thích sâu hơn.

1. Definition — Khái niệm là gì?

Nêu định nghĩa chính xác, mục đích và vấn đề mà khái niệm giải quyết. Phân biệt với khái niệm gần giống để tránh trả lời chung chung.

2. Mechanism — Nó hoạt động như thế nào?

Giải thích luồng hoạt động hoặc các thành phần tham gia. Với Java, có thể nói về ngữ nghĩa ngôn ngữ, runtime, memory model hoặc cấu trúc dữ liệu. Với Spring, làm rõ container, proxy, interceptor, transaction manager hoặc lifecycle khi phù hợp.

3. Example — Ví dụ thực tế hoặc code minh họa.

Đưa ví dụ ngắn, có thể dự đoán được. Nên có cả trường hợp đúng và trường hợp dễ gây hiểu nhầm. Nếu có thể, viết test để kiểm chứng.

4. Limitation — Những trường hợp không hoạt động như mong đợi.

Nêu điều kiện biên, giới hạn, lỗi phổ biến, giả định bị phá vỡ hoặc hành vi phụ thuộc phiên bản/cấu hình. Không khẳng định tuyệt đối khi hành vi còn phụ thuộc môi trường.

5. Trade-off — Khi nào nên hoặc không nên sử dụng?

So sánh với lựa chọn thay thế. Nêu chi phí, độ phức tạp, hiệu năng, khả năng bảo trì và bối cảnh phù hợp.

Mẫu ghi chú cho từng câu hỏi

Trường	Ghi chú của người học
Câu hỏi / Chủ đề	Ghi lại câu hỏi chính xác và nhóm kiến thức.
Definition	Định nghĩa trong 2–4 câu, dùng từ ngữ của bản thân.
Mechanism	Các bước hoạt động; thành phần tham gia; sơ đồ nếu cần.
Example	Ví dụ code, output dự đoán hoặc tình huống thực tế.
Limitation	Ít nhất một edge case, giới hạn hoặc lỗi thường gặp.
Trade-off	So sánh với tối thiểu một phương án thay thế.
Mức tự tin	0 = chưa hiểu; 1 = nhận diện; 2 = giải thích được; 3 = áp dụng/debug được.
Ngày ôn lại	Ghi ngày cần trả lời lại mà không xem ghi chú.

Quy tắc tự đánh giá

- Không tính là đã học xong nếu chỉ nhớ định nghĩa nhưng không giải thích được cơ chế.
- Nếu không có ví dụ, hãy tạo một ví dụ tối thiểu thay vì chỉ đọc thêm.
- Nếu câu trả lời thiếu phần Limitation hoặc Trade-off, đánh dấu để ôn lại.
- Phân biệt điều được Java/Spring specification bảo đảm với hành vi phụ thuộc implementation, database, cấu hình hoặc phiên bản.
- Không cần đưa đủ năm phần vào mọi câu trả lời phỏng vấn nếu câu hỏi đơn giản; khung này là công cụ luyện tập, không phải bài nói thuộc lòng.

PHẦN II — CÁCH SỬ DỤNG TÀI LIỆU

Mô hình 3 tầng

Tầng	Mục tiêu	Câu hỏi tự kiểm tra
Tầng 1 — Core Internals	Hiểu ngôn ngữ, runtime, framework internals và cơ chế nền tảng.	Vì sao nó hoạt động như vậy? Thành phần nào tham gia?
Tầng 2 — Production Engineering	Áp dụng vào hệ thống thực tế; xử lý lỗi, concurrency, hiệu năng và độ tin cậy.	Trong điều kiện tải, lỗi hoặc nhiều instance, giải pháp có còn đúng không?
Tầng 3 — Interview & System Reasoning	Giải thích, debug, so sánh phương án và bảo vệ quyết định.	Tôi có thể trình bày rõ ràng, có ví dụ và nêu trade-off không?

Nhịp học gợi ý

- Mỗi tuần: chọn một nhóm chủ đề chính, đọc tài liệu chính thức và ghi lại thuật ngữ cần hiểu.
- Mỗi ngày: học sâu một phần nhỏ, viết code tối thiểu và trả lời 5–10 câu không nhìn tài liệu.
- Cuối tuần: tổ chức một buổi mock interview, tự ghi âm hoặc nhờ người khác hỏi.
- Sau mock interview: viết lại các câu trả lời sai, thiếu cơ chế hoặc không nêu được giới hạn.
- Mỗi 2–3 tuần: quay lại các chủ đề cũ để kiểm tra khả năng nhớ dài hạn.

Tài liệu tham khảo chính

- Java: Dev.java — Learn Java
- Java Language Specification: JLS Java 21
- JVM Specification: JVMS Java 21
- Java API: Java 21 API documentation
- Spring Framework: Spring Framework Reference
- Spring Boot: Spring Boot Reference
- Spring Framework source: GitHub repository

Không cần đọc toàn bộ specification. Dùng tài liệu học có cấu trúc trước, sau đó tra cứu specification hoặc source code khi cần xác minh một hành vi cụ thể.

PHẦN III — JAVA CORE

Đề cương Java Core được chia thành ba tầng. Tầng 1 ưu tiên ngôn ngữ và runtime; Tầng 2 tập trung vào cách viết phần mềm bền vững; Tầng 3 chuyển kiến thức thành năng lực trả lời phỏng vấn và giải quyết vấn đề.

Tầng 1 — Java Language & JVM Internals

Mục tiêu: giải thích được ngữ nghĩa của Java, dự đoán hành vi code và hiểu những cơ chế runtime quan trọng.

1. OOP, type system và object model

- Encapsulation, abstraction, inheritance, polymorphism; composition so với inheritance.
- Interface, abstract class, default methods; dynamic dispatch và method resolution.
- Class, object, reference, object identity; stack frame và reference semantics ở mức khái niệm.
- == và equals(); hợp đồng equals/hashCode; symmetry, transitivity, consistency và null handling.
- String immutability, string pool, StringBuilder/StringBuffer và chi phí tạo object.
- static, final, this, super; access modifiers và phạm vi truy cập.
- Initialization order: static fields/blocks, instance fields/blocks, constructor, superclass/subclass.
- Nested class, static nested class, inner class, anonymous class và local class.
- Records, sealed classes và pattern matching: mục đích, giới hạn và bối cảnh dùng.

2. Generics

- Generic class, generic interface, generic method và type parameter bounds.
- Type erasure; điều gì còn và không còn ở runtime; hạn chế tạo generic array.
- Raw types, unchecked warning và heap pollution.
- Wildcards: ?, ? extends T, ? super T; PECS (Producer Extends, Consumer Super).
- Invariance của generic types; vì sao List không phải List.
- Type inference, diamond operator và target typing.
- Bridge methods ở mức khái niệm và ảnh hưởng của type erasure tới overriding.
- Giới hạn của generics với primitive types, instanceof và overloaded methods.

3. Collections Framework

- Collection hierarchy: List, Set, Queue, Deque, Map và khác biệt về contract.
- ArrayList: dynamic array, amortized cost, capacity và chi phí chèn/xóa.
- LinkedList: node-based structure, chi phí truy cập và các tình huống ít phù hợp.
- HashMap: hash, bucket, collision, equals/hashCode, resizing và load factor.
- HashSet và quan hệ với HashMap; uniqueness dựa trên equality.
- TreeMap/TreeSet: ordering, Comparator, Comparable và cây cân bằng ở mức khái niệm.
- Queue, Deque, PriorityQueue; FIFO, LIFO và priority ordering.
- Iterator, fail-fast behavior, ConcurrentModificationException và các hiểu nhầm phổ biến.
- Collections.unmodifiableX, List.of, Map.of và khác biệt giữa unmodifiable và immutable object.
- Chọn cấu trúc dữ liệu theo workload, complexity, memory overhead và semantics.

4. Exceptions và resource management

- Throwable hierarchy: Error, Exception, RuntimeException; checked và unchecked exceptions.
- try/catch/finally, multi-catch, exception chaining và cause.
- Throw vs throws; tạo custom exception; giữ nguyên nguyên nhân lỗi.
- Try-with-resources, AutoCloseable và suppressed exceptions.
- Return hoặc throw trong finally và vì sao nên tránh.
- Exception handling ở boundary: chuyển lỗi kỹ thuật thành lỗi phù hợp mà không mất thông tin.
- Không dùng exception làm luồng điều khiển thông thường khi có lựa chọn rõ ràng hơn.
- Null handling, Optional và cách tránh lạm dụng Optional trong field/parameter.

5. Functional Java và Stream API

- Functional interface, lambda, method reference và target type.

- Predicate, Function, Consumer, Supplier và các primitive specializations.
- Stream pipeline: source, intermediate operations, terminal operations và laziness.
- map, flatMap, filter, reduce, collect, groupingBy, partitioningBy.
- Stateless/stateful operations, encounter order và side effects.
- Stream so với vòng lặp truyền thống; readability, debugging và performance.
- Parallel streams: common ForkJoinPool, splitting, combining và giới hạn.
- Optional: map, flatMap, orElse, orElseGet, orElseThrow và eager/lazy evaluation.

6. JVM và runtime

- Vai trò của JDK, JRE (khái niệm), JVM và Java platform.
- Class loading: loading, linking, initialization; class loader delegation.
- Bytecode, javap và mối liên hệ giữa source code với bytecode.
- Runtime data areas: heap, Java stacks, PC registers, native method stacks, method area/metaspace.
- Object allocation, reachability, reference types và garbage collection ở mức khái niệm.
- JIT compilation, warm-up, profiling và vì sao benchmark đơn giản dễ sai.
- GC goals, pauses, throughput, latency và cách đọc log/metrics ở mức cơ bản.
- Memory leak trong Java: object vẫn reachable nhưng không còn cần thiết.
- Heap dump, thread dump và công cụ quan sát JVM ở mức nhập môn.
- Phân biệt hành vi được specification bảo đảm với chi tiết implementation của JVM.

7. Modern Java và API design

- Enum, annotations, var, records, sealed classes và pattern matching phù hợp phiên bản.
- Date/Time API: Instant, LocalDate, LocalDateTime, ZonedDateTime, Duration và Period.
- BigDecimal, rounding modes và lỗi thường gặp khi biểu diễn tiền tệ.
- API design: cohesion, coupling, invariants, defensive copying và immutability.
- Comparable và Comparator; contract của comparator và sắp xếp ổn định.
- Serialization basics, versioning và rủi ro khi deserialize dữ liệu không tin cậy.
- Modules và service loading ở mức nhận biết khi dự án có sử dụng.
- Backward compatibility, deprecation và cách chọn API phù hợp phiên bản Java.

Tầng 2 — Production Java Engineering

Mục tiêu: viết Java có thể vận hành được, xử lý tải đồng thời, lỗi và tài nguyên một cách có kiểm soát.

1. Collections, complexity và algorithmic cost

- Độ phức tạp thời gian/bộ nhớ; average, amortized và worst-case complexity.
- Chọn List/Set/Map/Queue theo pattern truy cập và yêu cầu semantics.
- Hash collision, resizing, capacity planning và allocation pressure.
- Comparator consistency, ordering và tác động tới TreeMap/TreeSet.
- Defensive copying, immutable snapshots và dữ liệu chia sẻ giữa các tầng.

2. Concurrency và Java Memory Model

- Process vs thread; platform thread, lifecycle và interruption.
- Race condition, atomicity, visibility, ordering và data race.
- Java Memory Model; happens-before, monitor lock và volatile semantics.
- synchronized, intrinsic locks, ReentrantLock, tryLock và lock discipline.
- AtomicInteger, AtomicLong, AtomicReference, CAS và giới hạn của atomic operations.
- Deadlock, livelock, starvation, contention và cách nhận diện.
- ConcurrentHashMap, BlockingQueue, CopyOnWriteArrayList và lựa chọn theo workload.
- ExecutorService, thread pool sizing, queue, rejection policy và shutdown.
- Future, CompletableFuture, composition, exception handling, timeout và cancellation.
- Virtual threads: use cases, blocking I/O, chi phí thread và những giới hạn cần kiểm chứng theo phiên bản.
- Thread safety, immutability, confinement và chia sẻ mutable state.
- Không nhầm thread safety trong một JVM với tính nhất quán phân tán giữa nhiều process.

3. I/O, network và quản lý tài nguyên

- java.io và java.nio; stream vs channel, buffer và selector ở mức phù hợp.
- Buffering, encoding, charset và xử lý dữ liệu lớn.
- File/path API, try-with-resources và xử lý lỗi I/O.
- Socket/network timeout, retry boundary và đóng tài nguyên.
- Serialization, format dữ liệu, compatibility và kiểm soát input không tin cậy.
- Resource exhaustion: file descriptor, connection, thread và memory.

4. Testing, debugging và performance

- Unit test, integration test, test doubles và boundary testing.
- Thiết kế test cho contract, null, empty, invalid input và exception.
- Kiểm thử concurrency: latch/barrier, repeated execution và giới hạn của test không deterministic.
- Logging có ngữ cảnh; phân biệt symptom với root cause.
- JFR, profiler, heap dump, thread dump và GC logs ở mức thực hành.
- Benchmark bằng JMH khi cần đo vi mô; tránh kết luận từ một lần chạy System.nanoTime.
- Phân tích allocation, GC pressure, contention và latency distribution.
- Refactoring an toàn, API compatibility và kiểm tra regression.

5. Secure and maintainable Java

- Input validation, output encoding và không tin cậy dữ liệu deserialize.
- Quản lý secrets, không ghi dữ liệu nhạy cảm vào log.
- Exception message không để lộ thông tin nội bộ không cần thiết.
- Dependency/version management và cập nhật thư viện.
- Immutability, clear ownership và API contract để giảm bug.
- Code review: correctness, concurrency, resource cleanup, testability và complexity.

Tầng 3 — Interview & Problem Solving

- Conceptual questions: định nghĩa, cơ chế, ví dụ, giới hạn và trade-off.
- Code reasoning: dự đoán output, exception, mutation, dispatch, collection behavior và concurrency behavior.
- Debugging: xác định nguyên nhân từ stack trace, log, heap/thread dump hoặc test tái hiện.
- Design questions: lựa chọn abstraction, collection, concurrency primitive, resource model và API contract.
- Phân tích complexity và lý do chọn cấu trúc dữ liệu; kết nối với DSA nhưng không đồng nhất hai mục tiêu.
- Trình bày câu trả lời có cấu trúc, nêu giả định và không khẳng định quá mức khi phụ thuộc JVM/implementation.

Bài tập thực hành Java Core

1. Viết test cho equals/hashCode và dùng object làm key trong HashMap; thử thay đổi field tham gia hash.
2. So sánh ArrayList, LinkedList, HashSet và TreeSet với các workload khác nhau; ghi lại giả định và complexity.
3. Tạo ví dụ generics dùng extends/super; giải thích vì sao một số phép gán bị compiler từ chối.
4. Viết pipeline Stream có grouping, flatMap và collector; so sánh với vòng lặp dễ đọc hơn.
5. Tạo race condition với counter; sửa bằng synchronized và AtomicInteger; nêu khác biệt.
6. Dùng ExecutorService xử lý batch công việc với giới hạn thread; xử lý exception và shutdown đúng.
7. Dùng CompletableFuture kết hợp hai tác vụ, timeout và exception recovery.
8. Tạo memory leak có chủ đích bằng collection giữ reference; quan sát heap dump/profiler nếu có công cụ.
9. Viết test cho Date/Time và BigDecimal để tránh lỗi timezone/rounding.
10. Tổ chức một buổi giải thích 10 câu Java Core theo khung năm bước.

PHẦN IV — SPRING CORE

Phần này tập trung vào Spring Framework Core và các tính năng liên quan trực tiếp. Spring Boot được đưa vào khi cần giải thích cách ứng dụng cấu hình và khởi động, nhưng không đồng nhất Boot với Framework.

Tầng 1 — Spring Framework Internals

Mục tiêu: hiểu container, bean lifecycle, proxy và các cơ chế mà nhiều tính năng Spring dựa vào.

1. IoC, DI và container

- IoC và DI: vấn đề giải quyết, constructor/setter/field injection và testability.
- BeanFactory vs ApplicationContext; các khả năng bổ sung của ApplicationContext.
- BeanDefinition: metadata bean, nguồn đăng ký và vai trò trong container.
- Component scanning, @Component, @Service, @Repository và stereotype annotations.
- @Configuration và @Bean; Java configuration so với component scanning.
- Constructor selection, dependency resolution và required dependencies.
- @Primary, @Qualifier, bean name và nhiều implementation cùng type.
- Optional dependencies, ObjectProvider và lazy resolution khi phù hợp.
- Circular dependencies: nguyên nhân, giới hạn và vì sao constructor cycle thường không giải quyết được.
- Profiles, conditional registration và ranh giới giữa Framework features với Boot auto-configuration.

2. Bean lifecycle và scope

- Quy trình đăng ký bean definition, instantiation, population/injection, initialization và destruction.
- BeanFactoryPostProcessor vs BeanPostProcessor; tác động và thời điểm chạy.
- InstantiationAwareBeanPostProcessor ở mức khái niệm.
- @PostConstruct, InitializingBean, custom init method và thứ tự callback.
- @PreDestroy, DisposableBean, custom destroy method và graceful shutdown.
- Singleton, prototype, request, session và application scope.
- Prototype injected into singleton; cách xử lý lookup/proxy khi cần instance mới.
- Lazy initialization và ảnh hưởng tới thời điểm phát hiện lỗi.
- FactoryBean và bean do FactoryBean tạo ra.
- Bean lifecycle với AOP proxy; nhận diện khi object được wrap hoặc thay thế.

3. AOP và proxy

- AOP terminology: aspect, advice, pointcut, join point, target và proxy.
- JDK dynamic proxy vs CGLIB proxy; điều kiện và giới hạn.
- @Before, @After, @AfterReturning, @AfterThrowing và @Around.
- Pointcut expressions và phạm vi interception.
- Self-invocation và lời gọi không đi qua proxy.
- Final/private method, interface exposure và các giới hạn proxy-based AOP.
- Advice ordering và tương tác giữa nhiều aspect.
- Proxy creation trong container; BeanPostProcessor và auto-proxy creator ở mức khái niệm.
- Spring AOP so với AspectJ weaving.
- Ứng dụng AOP: transaction, logging, metrics và cross-cutting concerns; tránh lạm dụng.

4. Transaction management

- Declarative transaction management và vai trò của TransactionInterceptor.
- PlatformTransactionManager và các khái niệm transaction status/synchronization.
- @Transactional: proxy, transaction boundary và method invocation.
- Propagation: REQUIRED, REQUIRES_NEW, SUPPORTS, NOT_SUPPORTED, MANDATORY, NEVER, NESTED.
- Isolation level và quan hệ với database transaction semantics.
- Rollback rules: unchecked/checked exception và cấu hình rollback.
- readOnly, timeout, transaction manager selection và các giả định cần kiểm chứng.
- Self-invocation, private/final method và transaction không được áp dụng như mong đợi.
- Transaction lồng nhau, suspended transaction và connection/resource implications.

- Transaction boundary ở service layer; không kéo dài transaction qua network call không cần thiết.
- Reactive transaction ở mức nhận biết nếu làm việc với reactive stack.

5. Events, resources và expression language

- ApplicationEventPublisher, ApplicationListener và @EventListener.
- Event synchronous/asynchronous; thứ tự, lỗi và giới hạn của event nội bộ.
- Transactional event listeners và thời điểm chạy theo transaction phase.
- Resource abstraction: classpath, filesystem, URL và resource loading.
- Environment, PropertySource, profiles và property resolution.
- SpEL: use cases, expression evaluation và cẩn trọng khi expression phụ thuộc input.
- Type conversion, Formatter/Converter và dữ liệu cấu hình.
- Internationalization/message source ở mức nhận biết.
- ApplicationContext lifecycle và startup/shutdown hooks.

Tầng 2 — Production Behavior & Reliability

Mục tiêu: áp dụng cơ chế Spring vào hệ thống thật, nơi có database, concurrency, lỗi dependency và nhiều instance.

1. Configuration và Spring Boot integration

- Externalized configuration, profiles, @ConfigurationProperties và @Value.
- Auto-configuration, conditional annotations và auto-configuration back-off.
- Bean overriding, configuration precedence và tránh cấu hình khó dự đoán.
- Component scan boundaries, package structure và modular configuration.
- Startup failure diagnosis: missing bean, ambiguous dependency, circular dependency.
- Phân biệt Spring Framework API với Spring Boot convenience/configuration.

2. Async, scheduling và retry

- @Async, executor selection, thread pool sizing, queue và rejection behavior.
- Proxy limitation của @Async và xử lý lời gọi nội bộ.
- Exception handling với async method; Future/CompletableFuture return semantics.
- @Scheduled: cron, fixedRate, fixedDelay, initialDelay và timezone.
- Scheduler thread pool, overlapping work và thời gian chạy dài hơn chu kỳ.
- Multi-instance scheduling: duplicate execution, distributed lock hoặc database claim strategy.
- Retry semantics, retryable vs non-retryable errors, backoff, jitter và retry budget.
- Spring Retry và resilience support theo phiên bản; không giả định API giống nhau ở mọi phiên bản.
- Circuit breaker, timeout, bulkhead và rate limiter, thường qua Resilience4j.
- Idempotency, duplicate execution và retry an toàn cho side effects.

3. Data access, concurrency và reliability

- Transaction boundary và cách phối hợp với repository/data access.
- Race condition giữa request, scheduled job và background worker.
- Optimistic/pessimistic locking và constraint ở database; giới hạn của lock trong JVM.
- Application events so với message broker; phạm vi đảm bảo của từng cơ chế.
- Kafka/RabbitMQ integration ở mức kiến trúc: delivery, retry, duplicate, dead-letter.
- Outbox pattern và tính nhất quán giữa database commit với phát message ở mức khái niệm.
- Graceful shutdown, in-flight work và khả năng mất công việc.
- Observability: structured logs, metrics, tracing và correlation identifiers.

4. Testing Spring applications

- Unit test thuần Java vs integration test khởi tạo Spring context.
- Spring TestContext, context caching và test lifecycle.
- @SpringBootTest và test slices; chọn test scope phù hợp.
- Kiểm tra DI, lifecycle, AOP proxy và transactional behavior bằng test.
- Testcontainers cho database/broker khi cần kiểm tra integration thực tế.
- Testing asynchronous behavior, scheduler và retry; tránh test dựa vào sleep tùy tiện.

- Test isolation, transaction rollback trong test và các khác biệt với production.
- Debug context loading, conditional bean registration và configuration.

Tầng 3 — Interview & System Reasoning

- Conceptual: giải thích container, lifecycle, proxy, transaction và configuration.
- Code reasoning: dự đoán hành vi khi self-invocation, nhiều bean cùng type, async exception hoặc transaction rollback.
- Debugging: tìm nguyên nhân từ startup logs, bean creation errors, proxy behavior và transaction boundary.
- Design: chọn giữa @Scheduled, Quartz và message broker theo yêu cầu độ bền, lịch và phân tán.
- Reliability: phân tích retry, idempotency, duplicate execution, locking và failure recovery.
- Trade-off: nêu rõ độ phức tạp vận hành, độ tin cậy, khả năng mở rộng và chi phí bảo trì.
- Version awareness: xác minh API/tính năng theo Spring Framework/Spring Boot version thực tế.

Bài tập thực hành Spring Core

1. Dựng AnnotationConfigApplicationContext và quan sát bean registration/creation.
2. Tạo hai implementation cùng interface; kiểm tra lỗi ambiguity, @Primary và @Qualifier.
3. Viết BeanPostProcessor ghi lại tên bean và các mốc lifecycle.
4. Tạo singleton phụ thuộc prototype; giải thích vì sao reference không tự đổi mỗi lần gọi.
5. Viết logging aspect và so sánh lời gọi qua bean khác với self-invocation.
6. Tạo hai method transaction với REQUIRED/REQUIRES_NEW; viết integration test kiểm tra rollback.
7. Tạo cấu hình theo profile và property; kiểm tra bean được đăng ký theo từng môi trường.
8. Tạo scheduled task có thể chạy lặp; thiết kế thao tác idempotent và kiểm tra trạng thái trước khi cập nhật.
9. Tạo retry cho lỗi tạm thời; đảm bảo không retry lỗi validation và không lặp side effect không an toàn.
10. Viết integration test cho context startup, proxy behavior và transaction boundary.

PHẦN V — BÀI TẬP CỐT LÕI VÀ KẾ HOẠCH ÔN TẬP

Bài tập cốt lõi

1. Tạo ngân hàng khoảng 60–100 câu hỏi, phân nhóm rõ ràng giữa Java Core và Spring Core. Tài liệu này cung cấp 80 câu khởi đầu.
2. Mỗi ngày trả lời 5–10 câu mà không nhìn tài liệu. Nói thành tiếng hoặc viết câu trả lời theo khung năm bước.
3. Mỗi tuần thực hiện một buổi mock interview. Chọn câu hỏi ngẫu nhiên; giới hạn thời gian trả lời để luyện sự mạch lạc.
4. Ghi lại câu trả lời sai, thiếu cơ chế, thiếu ví dụ hoặc bỏ qua limitation/trade-off; viết lại bằng ngôn ngữ của mình.
5. Với câu hỏi cơ chế, tạo một ví dụ code hoặc test nhỏ để xác minh thay vì chỉ đọc đáp án.
6. Mỗi 2 tuần, làm lại các câu từng trả lời sai để kiểm tra khả năng nhớ dài hạn.
7. Không đánh giá tiến độ bằng số trang đã đọc; đánh giá bằng khả năng giải thích, viết test và debug.

Lịch luyện mẫu trong một tuần

Ngày	Việc cần làm	Sản phẩm đầu ra
Thứ 2	Học một chủ đề mới; đọc tài liệu chính thức.	Ghi chú khái niệm và thuật ngữ.
Thứ 3	Viết ví dụ code tối thiểu; kiểm tra trường hợp bình thường.	Code chạy được hoặc test.
Thứ 4	Thử edge cases, lỗi và giới hạn.	Danh sách failure modes.
Thứ 5	Trả lời 5–10 câu không nhìn tài liệu.	Bản trả lời nháp.
Thứ 6	Ôn lại câu sai; đọc sâu những phần chưa hiểu.	Bản sửa và câu hỏi cần làm rõ.
Thứ 7	Mock interview 30–60 phút.	Điểm mạnh, lỗ hổng và kế hoạch sửa.
Chủ nhật	Ôn nhẹ hoặc nghỉ; lên kế hoạch tuần sau.	Danh sách ưu tiên tiếp theo.

Rubric chấm câu trả lời

Điểm	Mô tả
0	Không biết hoặc trả lời sai bản chất.
1	Nhớ được định nghĩa nhưng không giải thích được cơ chế.
2	Giải thích đúng cơ chế nhưng thiếu ví dụ hoặc limitation.
3	Có cơ chế và ví dụ chính xác; nêu được ít nhất một giới hạn.
4	So sánh được trade-off, nêu giả định và áp dụng vào tình huống thực tế.

Mục tiêu gợi ý trước khi phỏng vấn: phần lớn câu hỏi trọng tâm đạt mức 3; các chủ đề cốt lõi của vị trí ứng tuyển nên đạt mức 4.

PHẦN VI — NGÂN HÀNG 80 CÂU HỎI PHỎNG VẤN

Các câu hỏi dưới đây chỉ gồm đề bài, chưa có đáp án theo yêu cầu. Hãy ghi câu trả lời riêng theo khung năm bước. Các câu hỏi được chia nhóm để dễ theo dõi tiến độ.

Java Core A — OOP, object model và ngôn ngữ

J01. Java truyền tham số theo value hay reference? Giải thích bằng ví dụ object.

J02. Phân biệt ==, equals() và hashCode(); hợp đồng giữa equals() và hashCode() là gì?

J03. Vì sao String immutable? Điều đó ảnh hưởng thế nào đến pooling, security và concurrency?

J04. Overloading khác overriding như thế nào? Khi nào method được chọn ở compile time và runtime?

J05. Interface khác abstract class ở đâu? Khi nào composition phù hợp hơn inheritance?

J06. Trình bày thứ tự khởi tạo static field/block, instance field/block và constructor giữa superclass/subclass.

J07. final áp dụng cho variable, method và class có ý nghĩa gì? final reference có làm object immutable không?

J08. Checked exception và unchecked exception khác nhau thế nào? Khi nào nên tạo custom exception?

J09. Optional nên được dùng ở đâu và khi nào việc dùng Optional làm API khó hiểu hơn?

J10. Record khác class thông thường thế nào? Record có tự động làm mọi object lồng bên trong immutable không?

Java Core B — Generics, Collections và Functional Java

J11. Type erasure là gì và nó tạo ra những giới hạn nào đối với generics?

J12. Giải thích PECS và phân biệt ? extends T với ? super T bằng ví dụ.

J13. Vì sao List<Integer> không phải subtype của List<Number>?

J14. HashMap tìm key như thế nào? Vai trò của hashCode(), equals(), collision và resizing là gì?

J15. Điều gì có thể xảy ra nếu một object được dùng làm key trong HashMap rồi thay đổi field tham gia hashCode()?

J16. ArrayList và LinkedList khác nhau thế nào về cấu trúc, complexity và workload phù hợp?

J17. HashSet và TreeSet khác nhau thế nào về uniqueness, ordering và chi phí thao tác?

J18. Comparable khác Comparator thế nào? Điều gì xảy ra nếu comparator không nhất quán?

J19. Stream API lazy evaluation là gì? Phân biệt intermediate operations và terminal operations.

J20. Parallel stream có thể chậm hơn stream tuần tự trong trường hợp nào? Các vấn đề về shared state là gì?

Java Core C — JVM, memory và concurrency

J21. Phân biệt JDK, JVM và Java runtime; source code được thực thi như thế nào ở mức tổng quan?

J22. Class loading, linking và initialization là gì? Class loader delegation có mục đích gì?

J23. Heap, Java stack và metaspace chứa những loại dữ liệu nào ở mức khái niệm?

J24. Garbage Collector xác định object có thể thu hồi như thế nào? Vì sao không nên dựa vào GC để đóng resource?

J25. Memory leak trong Java có thể xảy ra dù có garbage collector không? Hãy nêu một ví dụ.

J26. Phân biệt atomicity, visibility và ordering trong concurrency.

J27. volatile bảo đảm điều gì và không bảo đảm điều gì? Vì sao volatile không làm phép i++ trở thành atomic?

J28. synchronized hoạt động thế nào và nó liên hệ ra sao với happens-before?

J29. Race condition, deadlock, livelock và starvation khác nhau thế nào?

J30. AtomicInteger/CAS khác synchronized ở đâu? Khi nào lựa chọn này không giải quyết được vấn đề tổng thể?

Java Core D — Production, API và debugging

J31. ExecutorService giải quyết vấn đề gì so với tự tạo Thread cho mỗi task? Cần cấu hình và shutdown như thế nào?

J32. Future khác CompletableFuture thế nào? Làm sao kết hợp nhiều task và xử lý exception?

J33. Platform thread khác virtual thread ở đâu? Khi nào virtual thread có lợi và cần kiểm tra những giới hạn nào?

J34. ConcurrentHashMap khác HashMap thế nào? Thread-safe collection có làm mọi chuỗi thao tác trở thành atomic không?

J35. Try-with-resources hoạt động như thế nào? Suppressed exception là gì?

J36. Instant, LocalDateTime và ZonedDateTime khác nhau thế nào? Vì sao thời điểm thực tế thường cần timezone/offset rõ ràng?

J37. Vì sao nên dùng BigDecimal cho phép tính tiền và cần chú ý gì về rounding/scale?

J38. Benchmark bằng cách gọi System.nanoTime() một lần có thể gây hiểu lầm như thế nào?

J39. Khi ứng dụng tăng memory hoặc CPU bất thường, bạn sẽ thu thập bằng chứng và điều tra theo thứ tự nào?

J40. Thiết kế một API Java immutable: cần quan tâm đến constructor, defensive copying, nested mutable objects và equality như thế nào?

Ngân hàng câu hỏi — Spring Core

Spring Core A — IoC, DI và Bean lifecycle

S01. IoC khác DI thế nào? Spring container giải quyết vấn đề gì?

S02. BeanFactory và ApplicationContext khác nhau ở đâu?

S03. BeanDefinition là gì? Spring biết và đăng ký bean từ những nguồn nào?

S04. @Component khác @Bean như thế nào? Khi nào nên dùng mỗi cách?

S05. @Primary và @Qualifier giải quyết vấn đề gì khi có nhiều bean cùng type?

S06. Trình bày các giai đoạn chính trong bean lifecycle và vai trò của BeanPostProcessor.

S07. BeanFactoryPostProcessor khác BeanPostProcessor thế nào về mục đích và thời điểm tác động?

S08. Singleton scope có đồng nghĩa với thread-safe không? Prototype scope có nghĩa gì khi inject vào singleton?

S09. Circular dependency là gì? Vì sao constructor injection có thể làm dependency cycle không thể khởi tạo?

S10. @Lazy ảnh hưởng thế nào tới thời điểm khởi tạo và thời điểm phát hiện lỗi?

Spring Core B — AOP, proxy và transaction

S11. AOP giải quyết loại vấn đề nào? Giải thích aspect, advice, pointcut, join point và target.

S12. JDK Dynamic Proxy khác CGLIB proxy thế nào? Các giới hạn quan trọng là gì?

S13. Vì sao self-invocation có thể làm @Transactional hoặc @Async không hoạt động như mong đợi?

S14. @Around advice có thể thay đổi kết quả method hoặc xử lý exception như thế nào?

S15. Spring declarative transaction hoạt động qua những thành phần nào?

S16. REQUIRED khác REQUIRES_NEW thế nào? Khi nào transaction hiện tại bị suspend?

S17. Những loại exception nào mặc định khiến transaction rollback? Làm sao cấu hình khác đi?

S18. Isolation level và propagation khác nhau thế nào? Chúng giải quyết hai loại vấn đề gì?

S19. Vì sao gọi một HTTP API chậm bên trong transaction có thể gây vấn đề?

S20. Spring AOP proxy khác AspectJ weaving thế nào? Khi nào cần hiểu sự khác biệt này?

Spring Core C — Configuration, async và scheduling

S21. Spring Boot auto-configuration hoạt động dựa trên những điều kiện nào? Nó khác Spring Framework Core thế nào?

S22. @Value và @ConfigurationProperties khác nhau thế nào khi quản lý cấu hình?

S23. Profile và conditional bean registration giải quyết vấn đề gì? Làm sao kiểm tra bean nào được đăng ký?

S24. Application event đồng bộ khác asynchronous event thế nào? Những giới hạn của event nội bộ là gì?

S25. @Async sử dụng cơ chế nào để thực thi task? Làm sao cấu hình executor và xử lý exception?

S26. @Scheduled fixedRate khác fixedDelay thế nào? Cần lưu ý gì khi task chạy lâu hơn chu kỳ?

S27. Vì sao @Scheduled có thể chạy cùng một công việc trên nhiều instance? Có thể giảm rủi ro bằng những cách nào?

S28. Retry nên áp dụng cho loại lỗi nào? Vì sao retry thao tác ghi có thể gây side effect trùng lặp?

S29. Circuit breaker khác retry thế nào? Khi kết hợp hai cơ chế, cần chú ý điều gì?

S30. Khi nào chọn @Scheduled, Quartz hoặc message broker cho công việc nền? Hãy nêu trade-off.

Spring Core D — Testing, reliability và debugging

S31. Unit test thuần Java khác Spring integration test thế nào? Khi nào cần khởi tạo ApplicationContext?

S32. @SpringBootTest và test slices khác nhau thế nào về phạm vi và chi phí?

S33. Spring TestContext cache application context như thế nào ở mức khái niệm? Điều gì có thể làm test khó cô lập?

S34. Làm sao viết test chứng minh một method thực sự chạy qua transactional proxy?

S35. Testcontainers giải quyết vấn đề gì so với mock repository hoặc dùng database H2?

S36. Làm thế nào kiểm thử retry/backoff mà không dựa vào sleep dài và test thiếu ổn định?

S37. Làm sao thiết kế scheduled job idempotent khi job có thể được chạy nhiều lần?

S38. Trong hệ thống nhiều instance, tại sao synchronized hoặc ReentrantLock trong JVM không đủ để khóa công việc toàn hệ thống?

S39. Phân biệt application event với Kafka/RabbitMQ message trong phạm vi durability và coupling.

S40. Khi Spring Boot không khởi động vì missing/ambiguous bean hoặc circular dependency, bạn sẽ debug theo quy trình nào?

PHẦN VII — CHECKLIST TỰ ĐÁNH GIÁ

Java Core

- ☐ Tôi giải thích được equals/hashCode và dự đoán được hành vi của HashMap.
- ☐ Tôi hiểu generics, wildcards, PECS và type erasure.
- ☐ Tôi chọn được collections dựa trên contract, complexity và workload.
- ☐ Tôi giải thích được class loading, heap/stack, GC và memory leak ở mức phù hợp.
- ☐ Tôi phân biệt atomicity, visibility, ordering và hiểu happens-before.
- ☐ Tôi có thể viết, kiểm thử và debug code concurrent cơ bản.
- ☐ Tôi biết cách quản lý tài nguyên, exception và API contract.
- ☐ Tôi biết khi nào cần profiler/heap dump/thread dump thay vì phỏng đoán.

Spring Core

- ☐ Tôi giải thích được IoC, DI, BeanFactory, ApplicationContext và BeanDefinition.
- ☐ Tôi mô tả được bean lifecycle, post-processors và scope.
- ☐ Tôi hiểu proxy, AOP, self-invocation và các giới hạn liên quan.
- ☐ Tôi giải thích được transaction boundary, propagation, isolation và rollback.
- ☐ Tôi phân biệt Spring Framework với Spring Boot auto-configuration.
- ☐ Tôi hiểu async, scheduling, retry và các vấn đề nhiều instance.
- ☐ Tôi biết cách kiểm thử bean wiring, proxy, transaction và background tasks.
- ☐ Tôi có thể phân tích reliability, idempotency và failure recovery.

Interview readiness

- ☐ Tôi trả lời được 5–10 câu mỗi ngày mà không nhìn tài liệu.
- ☐ Tôi thực hiện ít nhất một mock interview mỗi tuần.
- ☐ Tôi ghi lại câu trả lời sai/thiếu cơ chế và viết lại.
- ☐ Tôi có ví dụ code hoặc test cho các khái niệm trọng tâm.
- ☐ Tôi nêu được limitation và trade-off, không chỉ định nghĩa.
- ☐ Tôi kiểm tra phiên bản Java/Spring và không khẳng định API phụ thuộc phiên bản khi chưa xác minh.

Kế hoạch ôn lại cá nhân

Chủ đề	Điểm hiện tại (0–4)	Lỗ hổng cần sửa	Ngày ôn lại
Java — OOP/Object model			
Java — Generics/Collections			
Java — JVM/Memory			
Java — Concurrency			
Spring — IoC/Lifecycle			
Spring — AOP/Proxy			
Spring — Transactions			
Spring — Async/Reliability			
Interview reasoning			

Kết thúc tài liệu. Hãy cập nhật checklist theo tiến độ thực tế; ưu tiên hiểu sâu các chủ đề thường gặp trong JD và các vấn đề bạn đã gặp khi xây dựng project.